Mathematical Modeling

Modeling Patterns: Blending, Networks, Objectives, and Goals

Personal study notes

Lecture material Cream boxes mark content confirmed in supplied current lecture sources. Highlighting identifies recorded statements or models, not every surrounding proof or an examination requirement. Source details remain in the final remarks.

Contents

1 Building an LP from a statement **1**

2 Blending: a ratio that yields linear constraints **2**
2.1 Decisions, units, and conservation . . . 2
2.2 Why multiplying preserves the quality requirement . . . 2
2.3 Eliminating a variable and proving the optimum . . . 3

3 Transportation with exact demand and unavailable routes **3**
3.1 A complete small instance and a lower-bound certificate . . . 3

4 Production with storage as a forward-time network **4**
4.1 Why the transportation costs contain time . . . 4
4.2 An optimal schedule without a black-box solve . . . 4

5 Minimum-cost flow and the meaning of signs **5**
5.1 Cuts detect delivery bottlenecks . . . 5
5.2 Why some continuous network models have integer optima . . . 6

6 Choosing an objective and comparing several goals **6**
6.1 Constants, feasibility, and genuine decisions . . . 6
6.2 Weights, thresholds, and max–min reformulation . . . 7

7 Soft constraints: solving a deliberately changed model **8**
7.1 What the deviation variables measure . . . 8
7.2 A hand comparison of two relaxation criteria . . . 8

8 Questions for checking understanding **8**

9 Sources and editorial remarks **9**

1 Building an LP from a statement

A linear model expresses decisions using fixed coefficients, additive contributions, and explicitly stated domains. A useful formulation requires more than recognizing a familiar matrix: explain why a capacity is an inequality, a balance is an equality, and an auxiliary variable preserves the intended criterion. The examples below use continuous nonnegative quantities unless a different domain is stated. All solutions are justified by hand bounds before numerical checks.

2 Blending: a ratio that yields linear constraints

2.1 Decisions, units, and conservation

Choose monthly tonnes $x_1, \ldots, x_5 \geq 0$ of five input oils and tonnes $y \geq 0$ of finished oil. Assume no refining mass loss, negligible refining cost, and a quality measure that is a mass-weighted average. The data are

Input	VEG1	VEG2	OIL1	OIL2	OIL3
Purchase cost (PLN/t)	110	120	130	110	115
Hardness	8.8	6.1	2	4.2	5

Finished oil sells for 150 PLN/t. Vegetable refining capacity is 200 t/month; other-oil capacity is 250 t/month. Conservation and capacities give

$$y = \sum_{i=1}^{5} x_i, \qquad x_1 + x_2 \leq 200, \qquad x_3 + x_4 + x_5 \leq 250.$$

The equality asserts that every purchased tonne enters the finished product; it would require modification if some input were discarded or mass were lost. Revenue minus purchase cost is

$$\max \; 150y - 110x_1 - 120x_2 - 130x_3 - 110x_4 - 115x_5.$$

2.2 Why multiplying preserves the quality requirement

For $y > 0$, average hardness is

$$H = \frac{8.8x_1 + 6.1x_2 + 2x_3 + 4.2x_4 + 5x_5}{y}.$$

Requiring $3 \leq H \leq 6$ is equivalent, for positive y, to

$$3y \leq 8.8x_1 + 6.1x_2 + 2x_3 + 4.2x_4 + 5x_5 \leq 6y. \tag{1}$$

The directions are preserved because the multiplier y is positive. Every coefficient is fixed, so these are linear inequalities. Multiplication by an expression of unknown sign would need separate cases and would not justify this step as written.

If $y = 0$, conservation and $x \geq 0$ force $x = 0$. The average is undefined, but the linear inequalities accept this no-production plan. Allowing no production is a modeling convention. A delivery requirement $y \geq q > 0$ would exclude it; silently imposing $y > 0$ instead changes the feasible set and its closedness.

As a simple illustration, mix v tonnes of VEG1 and o tonnes of OIL2. The upper-quality bound becomes $2.8v \leq 1.8o$, or $v/o \leq 9/14$ when $o > 0$. An individual input above hardness 6 can therefore be part of an acceptable mixture.

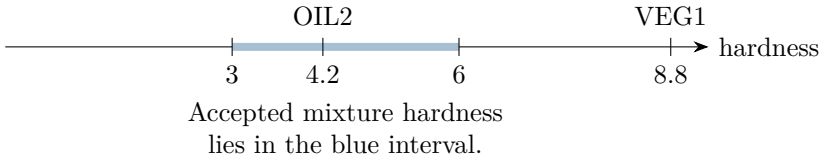


Figure 1: Weighted averages interpolate between component qualities. At the upper boundary the VEG1 mass fraction is 9/23.

2.3 Eliminating a variable and proving the optimum

Substitute $y = \sum_i x_i$ into profit. The margins become $(40, 30, 20, 40, 35)$ PLN/t. In the upper-quality inequality, subtract $6y$:

$$\frac{14}{5}x_1 + \frac{1}{10}x_2 - 4x_3 - \frac{9}{5}x_4 - x_5 \le 0.$$

An optimal plan is

$$x = \left(\frac{4300}{27}, \frac{1100}{27}, 0, 250, 0\right), \qquad y = 450. \tag{2}$$

Both refining capacities and the upper hardness bound are tight, giving $H = 6$; the lower bound is satisfied. Its profit is $475000/27$ PLN.

To prove optimality, weight vegetable capacity by $800/27$, other-oil capacity by $140/3$, and the upper-quality inequality by $100/27$. All weights are nonnegative. The combined left-side coefficients are

$$(40, 30, 860/27, 40, 1160/27),$$

each at least the corresponding margin. Thus every feasible plan satisfies

$$40x_1 + 30x_2 + 20x_3 + 40x_4 + 35x_5 \le \frac{800}{27}\,200 + \frac{140}{3}\,250 = \frac{475000}{27}.$$

Plan (2) attains the bound. Strict excess coefficients for x_3, x_5 force them to zero at every optimum; the three tight weighted rows then determine the other amounts uniquely. The proof also explains why buying the cheapest input independently of mixture quality can miss the optimum.

3 Transportation with exact demand and unavailable routes

Let $x_{ij} \ge 0$ be units sent from supplier i to customer j. Use a variable only for an allowed route $(i, j) \in E$. A prohibited route is absent, or its flow is fixed to zero; an infinite cost is a shorthand for prohibition, not a finite coefficient to pass to an LP solver.

$$\min \sum_{(i,j)\in E} c_{ij}x_{ij}, \qquad \sum_{j:(i,j)\in E} x_{ij} \le S_i, \qquad \sum_{i:(i,j)\in E} x_{ij} = D_j.$$

Supplier capacities are upper bounds. Customer demand is exact in this model; if oversupply were allowed, the appropriate customer condition could instead be a lower bound. Total capacity at least total demand is necessary, but route restrictions can make particular customer subsets impossible to serve.

3.1 A complete small instance and a lower-bound certificate

Take $S = (135, 56, 93)$ and $D = (62, 83, 39, 91)$. Costs per unit are

	Customer 1	Customer 2	Customer 3	Customer 4
Supplier 1	132	—	97	103
Supplier 2	85	91	—	—
Supplier 3	106	89	100	98

The following flow matrix satisfies demands and capacities, with dashes denoting unavailable routes:

$$X = \begin{pmatrix} 0 & — & 39 & 87 \\ 56 & 0 & — & — \\ 6 & 83 & 0 & 4 \end{pmatrix}.$$

Row totals are $(126, 56, 93)$; nine units of supplier 1 capacity are unused. The cost is 25919.

Choose customer values $v = (111, 94, 97, 103)$ and supplier charges $u = (0, 26, 5)$. For each allowed route, check $c_{ij} \geq v_j - u_i$. Multiplying by its nonnegative flow and summing gives

$$\sum c_{ij}x_{ij} \geq \sum_j v_j D_j - \sum_i u_i \sum_j x_{ij} \geq \sum_j v_j D_j - \sum_i u_i S_i = 25919.$$

The second inequality uses $u \geq 0$ and the supplier upper bounds. The proposed plan reaches this lower bound. Used routes have equality in the route bound, and each supplier with a positive charge uses its full capacity; these facts explain the certificate's equality conditions.

4 Production with storage as a forward-time network

4.1 Why the transportation costs contain time

Let $p_{k,d}$ and $p_{k,n}$ be day- and night-shift production in month k. Let $I_k \geq 0$ be stock after meeting month k's demand D_k. Assume no backlogging, no mass loss, unlimited storage, initially empty stock, and no desired final surplus: $I_0 = I_4 = 0$. Conservation gives

$$I_{k-1} + p_{k,d} + p_{k,n} - D_k = I_k. \tag{3}$$

The data are

Month	January	February	March	April
Day capacity	100	150	140	160
Night capacity	50	75	70	80
Demand	80	200	300	200

Day production costs 1 PLN/unit, night production 1.5, and each unit stored for one month costs 0.30. The objective is

$$\min \sum_{k=1}^{4} (p_{k,d} + 1.5p_{k,n}) + 0.30 \sum_{k=1}^{3} I_k,$$

with production between zero and its capacity.

Equivalently, let $x_{k,s,t}$ be production from shift s in month k used for demand in month t. Only $t \geq k$ is allowed, since later production cannot meet earlier demand. If shift s costs a_s, that route costs $a_s + 0.30(t - k)$ per unit. Supplier rows limit production and customer rows equal demand. Each unit assigned to a later month crosses $t - k$ storage intervals, proving the equivalence of route costs and inventory holding cost.

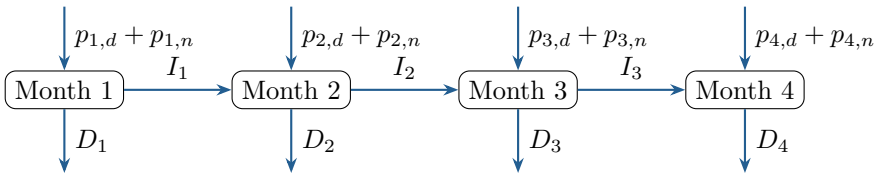


Figure 2: Stock carries material only forward. Each node's inflow and production equal its demand and outgoing stock.

4.2 An optimal schedule without a black-box solve

An optimal schedule is

Month	January	February	March	April
Day production	100	150	140	160
Night production	45	75	70	40
Closing inventory	65	90	0	0

Check each balance in (3). Total demand is 780, while total day capacity is 550. Any admissible schedule therefore needs at least 230 night units and production cost at least $780 + 0.5 \cdot 230 = 895$ PLN.

March demand is 300, but March capacity is 210, so $I_2 \geq 90$. Combining February and March balances with their capacities gives

$$I_1 \geq 200 + 300 - (225 + 210) = 65.$$

Other inventories are nonnegative. Holding cost is therefore at least $0.30(65 + 90) = 46.5$ PLN. The displayed schedule attains both bounds, giving the exact optimum $941.5 = 1883/2$ PLN. Enough total capacity alone would not prove feasibility: these prefix-demand arguments also respect time.

5 Minimum-cost flow and the meaning of signs

For a directed graph (V, E) choose flow $f_{uv} \geq 0$ on each arc, bounded by capacity C_{uv}. Let s_v be net supply: positive at a source and negative at a demand node. The model is

$$\min \sum_{(u,v)\in E} p_{uv} f_{uv}, \qquad 0 \leq f_{uv} \leq C_{uv}, \qquad \sum_{u:(u,v)\in E} f_{uv} + s_v = \sum_{w:(v,w)\in E} f_{vw}.$$

Incoming flow plus local production equals outgoing flow. Summing all node balances cancels every arc and gives $\sum_v s_v = 0$, a necessary condition. It is insufficient when capacities or missing routes block deliveries. With an incidence matrix having $+1$ at an arc's tail and -1 at its head, the same convention is $Bf = s$. Reversing the matrix convention also reverses the sign of the supply vector; keeping only one of these changes is an error.

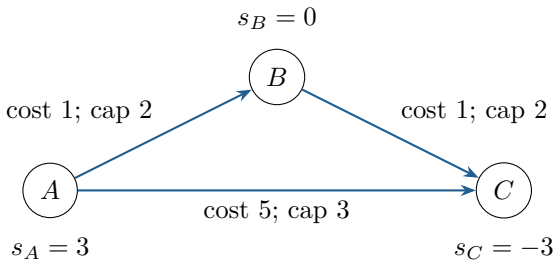


Figure 3: Three units must reach C. The cheap path has a bottleneck of two units.

Here balances force $f_{AB} = f_{BC} = t$ and $f_{AC} = 3 - t$, with $0 \leq t \leq 2$. Cost is $2t + 5(3 - t) = 15 - 3t$, minimized at $t = 2$ with value 9. This derivation simultaneously checks conservation, capacities and optimality.

5.1 Cuts detect delivery bottlenecks

For a node subset $U \subseteq V$, sum its balance equations. Internal arcs cancel, leaving

$$\sum_{v\in U} s_v = \sum_{(u,v):u\in U, v\notin U} f_{uv} - \sum_{(u,v):u\notin U, v\in U} f_{uv} \leq \sum_{(u,v):u\in U, v\notin U} C_{uv}.$$

The total net supply inside a cut cannot exceed its outgoing capacity. This necessary condition can expose infeasibility even when $\sum_v s_v = 0$.

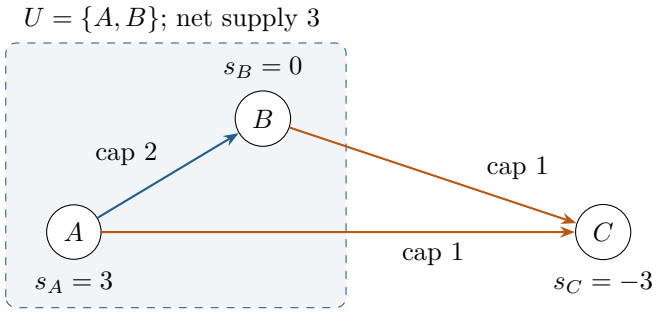


Figure 4: A different network with balanced supply and demand but insufficient outgoing cut capacity: $3 > 1 + 1$. No costs can repair this infeasibility.

For pure single-commodity flows with zero lower bounds, these inequalities for all subsets, together with total balance, are also sufficient; this is a form of the max-flow/min-cut theorem. Checking just one cut or total capacity does not establish sufficiency. Lower bounds require an adjusted balance model. To see the connection, add a super-source with an arc of capacity s_v to each supply node, and an arc of capacity $-s_v$ from each demand node to a super-sink. Put $S = \sum_{s_v > 0} s_v$. A source–sink cut containing original nodes U has capacity $S - \sum_{v \in U} s_v + C_{\text{out}}(U)$. Thus the subset inequalities say every such cut has capacity at least S. Max-flow/min-cut then supplies a flow of value S, saturating all supply and demand arcs and recovering the original balances.

5.2 Why some continuous network models have integer optima

Theorem 5.1 (Network integrality, stated without proof)**.** *If supplies and finite arc capacities are integers, every vertex of* $\{f : Bf = s,\ 0 \le f \le C\}$ *is integer. Thus a nonempty such network has an integer minimum-cost optimum, regardless of whether costs are integers.*

The structural reason is *total unimodularity*: every square submatrix of a directed incidence matrix has determinant 0, 1, or -1. Adding arc bounds preserves the required structure; integral right-hand sides then give integral vertices. This also explains integer transportation and assignment optima under integral data. Arbitrary LPs do not have this property. An interior optimal point can still be fractional: a convex combination of distinct integer optima need not be integer. The guarantee is the existence of an integer optimizer, not integrality of every feasible or optimal flow. Extra side constraints, such as shared resource budgets or coupling several commodities, can destroy the guarantee and require integer programming.

Node balances are also linearly dependent: each incidence column sums to zero. In a connected network one balance is redundant; in a graph with several connected components, each component must balance separately. Remove redundant rows before choosing a full-row-rank simplex basis.

6 Choosing an objective and comparing several goals

6.1 Constants, feasibility, and genuine decisions

An unavoidable fixed cost K changes profit $r^T x$ to $r^T x - K$. It changes the reported value but leaves the optimizers unchanged. A startup cost incurred only if production occurs is decision-dependent: modeling it normally introduces an activation decision, often a binary variable. It cannot be dropped as an unavoidable constant.

For one product with a justified finite upper bound $U > 0$, introduce $y \in \{0, 1\}$ and impose $0 \le x \le Uy$, with profit $px - Ky$, $K > 0$. Positive production forces $y = 1$ and incurs the

startup cost; $y = 0$ forces $x = 0$. If production is zero, maximizing profit chooses $y = 0$ when y has no other role. A minimum batch $\ell > 0$ can be enforced by $\ell y \le x$. Allowing $0 \le y \le 1$ instead creates a relaxation that can pay only part of the startup cost. The linking coefficient U should follow from a physical bound; an arbitrary huge number weakens that relaxation and harms scaling. This use of a large linking bound differs from an artificial-variable Big M penalty in simplex initialization.

If the question is simply whether requirements can be met, use an objective such as $\min 0$ over the original feasible set. Every feasible point then has equal objective value. Also, $x_1 + x_2 \le 2$, $x_1 \ge 1$, $x_2 \ge 1$ force $(x_1, x_2) = (1, 1)$ independently of the objective. Recognizing this singleton does not justify dropping sign or inequality constraints in a general model.

6.2 Weights, thresholds, and max–min reformulation

Suppose several affine criteria $f_i(x)$ are to be maximized. Their individual optima help reveal trade-offs, but may be incompatible. Three LP formulations are:

1. **Weighted sum:** maximize $\sum_i w_i f_i(x)$ with nonnegative weights. Scaling and units are part of the preference specification.

2. **Thresholds:** maximize one criterion subject to $f_i(x) \ge \beta_i$ for the others. Thresholds may make the model infeasible.

3. **Worst normalized achievement:** maximize $\min_i g_i(x)$, where the g_i are comparable affine normalized criteria. Introduce a free variable z and maximize z subject to $z \le g_i(x)$ for every i.

For fixed x, the largest feasible z is exactly $\min_i g_i(x)$, proving the last reformulation. Imposing $z \ge 0$ without justification could exclude valid decisions with negative scores. Positive weights ensure that an attained weighted-sum optimum is Pareto efficient: a feasible decision improving one criterion and worsening none would strictly improve the weighted sum.

For $0 \le x \le 1$, take $f_1 = x$ and $f_2 = 1 - x$. Equal weights make every point optimal. A greater weight on f_1 selects $x = 1$; a greater weight on f_2 selects $x = 0$. A threshold $f_2 \ge \beta$, $0 \le \beta \le 1$, gives $x^* = 1 - \beta$ when maximizing f_1. The max–min model has $z \le x$, $z \le 1 - x$, hence $2z \le 1$; $x = z = 1/2$ attains this bound. Replacing f_1 by $100x$ without adjusting normalization changes these choices.

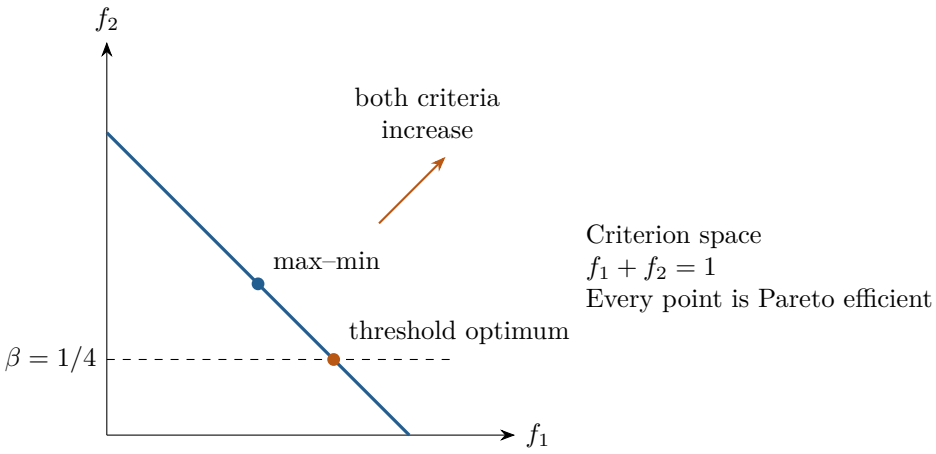


Figure 5: For $f_1 = x$, $f_2 = 1 - x$, improving either criterion worsens the other. Equal weights support the entire segment; a threshold or max–min rule selects a particular compromise. This is criterion space, not a two-variable feasible decision set.

With zero weights, a weighted optimum may ignore improvements in an unweighted criterion, so the positive-weight efficiency argument no longer applies. For a discrete or nonconvex attainable criterion set, varying positive weights can also miss Pareto efficient points. For example, if the only attainable pairs are $(0, 3)$, $(1, 1)$ and $(3, 0)$, all are efficient, but selecting $(1, 1)$ by weights $w_1, w_2 > 0$ would require both $w_1 \geq 2w_2$ and $w_2 \geq 2w_1$, which is impossible. Threshold methods can still select it.

7 Soft constraints: solving a deliberately changed model

7.1 What the deviation variables measure

Keep inviolable conditions in a hard feasible set. For a relaxable upper target $a^T x \leq b$, use $a^T x \leq b + u$, $u \geq 0$. For a lower target use $a^T x \geq b - v$, $v \geq 0$. For an equality target use $a^T x = b + u - v$ with $u, v \geq 0$. With positive penalties, minimizing these variables gives one-sided violations or, for the equality, $|a^T x - b|$.

With a strictly positive weighted-sum penalty, the equality deviations cannot both be positive at an optimum: reducing both by their minimum preserves the equality and reduces the sum penalty.

Use either a weighted sum of deviations or a variable $z \geq 0$ bounding all weighted deviations to minimize the largest one. Deviations in tonnes, hours and PLN are incomparable without units-aware weights or normalization. Conservation laws and domains should stay hard unless changing them has a deliberate meaning. If the hard set itself is empty, softening other targets cannot restore feasibility.

7.2 A hand comparison of two relaxation criteria

Let $x \geq 0$ be hard and request $x \leq 1$, $x \geq 3$. These targets are incompatible. Relax them to

$$x \leq 1 + u, \qquad x \geq 3 - v, \qquad u, v \geq 0.$$

Subtracting the inequalities gives $u + v \geq 2$. Minimizing $u + v$ has value 2, attained by every $1 \leq x \leq 3$ with $u = x - 1$, $v = 3 - x$. Minimizing z with $u, v \leq z$ gives $2z \geq u + v \geq 2$; the unique balanced answer is $x = 2$, $u = v = z = 1$. Weighting violations differently, such as minimizing $2u + v$, selects $x = 1$, $u = 0$, $v = 2$ with value 2. Thus the choice of violation measure determines what compromise means.

Phase I in simplex also uses nonnegative artificials, but its purpose is to test feasibility of the original hard equations. A positive *minimum* artificial sum, equivalently a negative maximum of minus that sum, certifies failure of that test. Goal programming instead accepts specified violations and reports a solution of a modified problem.

8 Questions for checking understanding

1. Why does multiplying a quality ratio by output preserve inequality directions?
2. What does the blending model mean at zero production?
3. Why should unavailable routes be omitted rather than assigned a huge cost?
4. Why is total production capacity insufficient to guarantee on-time delivery?
5. How do node-balance signs depend on the incidence-matrix convention?
6. What changes when an objective is multiplied by a conversion factor?
7. Which requirements can be relaxed without losing the physical meaning of a model?

9 Sources and editorial remarks

Lecture highlighting. Cream boxes identify selected statements, formulas and problem data recorded in the supplied current lecture sources. The course map in `docs/course-map-2026-27.md` gives the page destinations. Added hand proofs, qualifications and solution steps are not claimed as recorded lecture content.

The supplied current-course source is `ProgMat_w1.pdf`, 39 PDF pages, stored externally in `../lectures-2026-27/`. Its delivery date is not established. This document develops the modeling material in pp. 10–26: blending in Section 2, transportation in Section 3, storage in Section 4, flow in Section 5, objective selection in Section 6, and goals in Section 7. The supplied numerical data are retained. Hand optima, weighted bounds, the three-node example, normalized-goal example, relaxation example, and diagrams are added explanations. Continuous quantities, empty initial/final stock, no backlogging and unlimited storage are explicit modeling assumptions.

Network and trade-off extensions. Sections 5.1–5.2 and Figures 4 and 5 are added explanations; their current-course alignment is unknown. Network integrality was cross-checked against MIT OCW, *Networks 2: Network flows*, PDF pp. 30–33. The cut inequality, bottleneck and discrete Pareto counterexample are derived here. The max-flow/min-cut connection was checked against MIT OCW, *Network flows*, PDF pp. 2–3, Theorem 1 and Lemma 1. The startup formulation is an additional modeling explanation. Further study: max-flow/min-cut, total unimodularity and multiobjective optimization. These additions do not alter the numerical companion.

This conceptual supplement keeps the earlier LP geometry and proofs in Chapter 2 intact. The course map records reading routes and current task requirements separately from historical enrichment.

Numerical checks. The actual companion is modeling-patterns MATLAB script at `matlab/exercises/modeling_patterns_checks.m`. It checks the same factory, blending, shipment, storage, flow, regression, diet, objective and relaxation examples using base MATLAB and absolute tolerance 10^{-10}. Expected values include blending 475000/27, transportation 25919, storage 1883/2, and flow 9. Exact rational calculations independently checked the new hand results. The user supplied successful output on 2026-10-11 and confirmed MATLAB 26.2.0.3386108 (R2026b); all companion assertions passed. This run preceded the later diet-example revision in that companion. The execution record is `docs/matlab-verification.md`.